

```
import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.metrics import confusion_matrix, classification_report

from tensorflow import keras

from tensorflow.keras.datasets import imdb

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import ( Dense, Dropout, Embedding, GlobalMaxPooling1D )


# Vocabulary size

max_words = 10000


# Load IMDB dataset

(x_train, y_train), (x_test, y_test) = imdb.load_data(num_words=max_words)


# Maximum review length

max_sequence_length = 500


# Pad sequences

x_train = keras.preprocessing.sequence.pad_sequences(

    x_train,

    maxlen=max_sequence_length

)


x_test = keras.preprocessing.sequence.pad_sequences(

    x_test,

    maxlen=max_sequence_length

)


# Create model

model = Sequential()


model.add(Embedding(max_words, 100, input_length=max_sequence_length))

model.add(GlobalMaxPooling1D())

model.add(Dense(256, activation='relu'))
```

```
model.add(Dropout(0.5))

model.add(Dense(1, activation='sigmoid'))


# Compile model

model.compile( loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])


# Training parameters

batch_size = 64

epochs = 10


# Train model

history = model.fit( x_train, y_train, batch_size=batch_size, epochs=epochs, validation_data=(x_test, y_test) )


# Evaluate model

loss, accuracy = model.evaluate(x_test, y_test)

print(f"\nTest Accuracy: {accuracy:.4f}")


# Generate predictions

y_pred_probs = model.predict(x_test)

y_pred = (y_pred_probs > 0.5).astype(int)


# Plot predicted vs actual values

plt.figure(figsize=(10, 5))

plt.plot(y_test[:100], 'b.', label='Actual')

plt.plot(y_pred[:100], 'r.', label='Predicted')

plt.title('Actual vs Predicted Values')

plt.xlabel('Samples')

plt.ylabel('Sentiment')

plt.legend()

plt.show()

loss, accuracy = model.evaluate(x_test, y_test)

print("Test Loss:", loss)

print("Test Accuracy:", accuracy)


# Calculate metrics
```

```
cm = confusion_matrix(y_test, y_pred)

classification_rep = classification_report(y_test, y_pred)

# Plot confusion matrix

plt.figure(figsize=(8, 6))

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')

plt.title('Confusion Matrix')

plt.xlabel('Predicted Label')

plt.ylabel('True Label')

plt.show()

# Print classification report

print("Classification Report:\n", classification_rep)
```